

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Theory:

1. Blockchain Overview

Blockchain is a **distributed and decentralized ledger** that stores information in a series of linked blocks.

Each block contains:

- Transaction data
- Timestamp
- Previous block's hash
- Its own unique hash (digital fingerprint)

Once data is recorded in a blockchain, it becomes **immutable** because altering one block would require recalculating all subsequent blocks.

2. Mining

Mining is the process of:

1. Collecting pending transactions into a block.
2. Performing a computational puzzle (Proof-of-Work) to find a valid hash.
3. Adding the new block to the blockchain.
Broadcasting it to all connected peers.

Miners are rewarded with cryptocurrency for successfully mining a block.

3. Multi-Node Blockchain Network

In this lab, we simulate **three independent blockchain nodes** (5001, 5002, 5003).

Each node:

- Runs on a separate port.
- Maintains its own copy of the blockchain.
- Can connect with peers to share and validate blocks.

4. Consensus Mechanism

We use the **Longest Chain Rule**:

- If multiple versions of the chain exist, the **longest valid chain** is chosen.
- This ensures all nodes agree on a single transaction history.

5. Transactions & Mining Reward

Each transaction has:

- Sender

- Receiver
- Amount

When mining a block:

- Pending transactions are added to the block.
- A **reward transaction** is added automatically to pay the miner.

6. Chain Replacement

When `/replace_chain` is called:

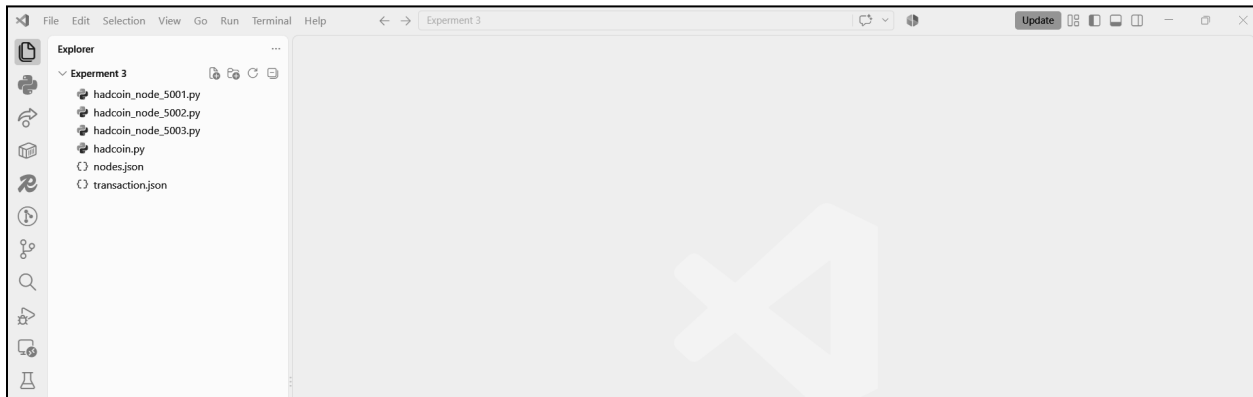
1. Node requests chains from peers.
2. If it finds a longer and valid chain, it replaces its own.
3. This keeps the blockchain consistent across all nodes.

Tools & Libraries Used

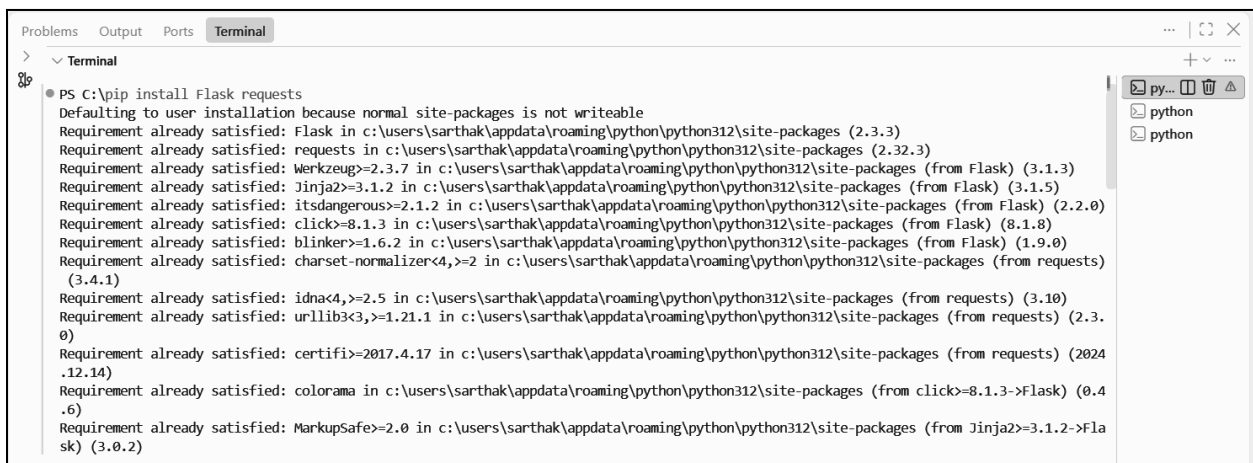
- **Python 3.x**
- **Flask** – Web framework for API endpoints
`pip install Flask`
- **Requests** – For HTTP communication between nodes
`pip install requests==2.18.4`
- **Postman** – For testing API requests
- Python Standard Libraries:
 - `datetime`
 - `jsonify`
 - `hashlib`
 - `uuid4`
 - `urlparse`
 - `request`

Step by Step Implementation and Output:-

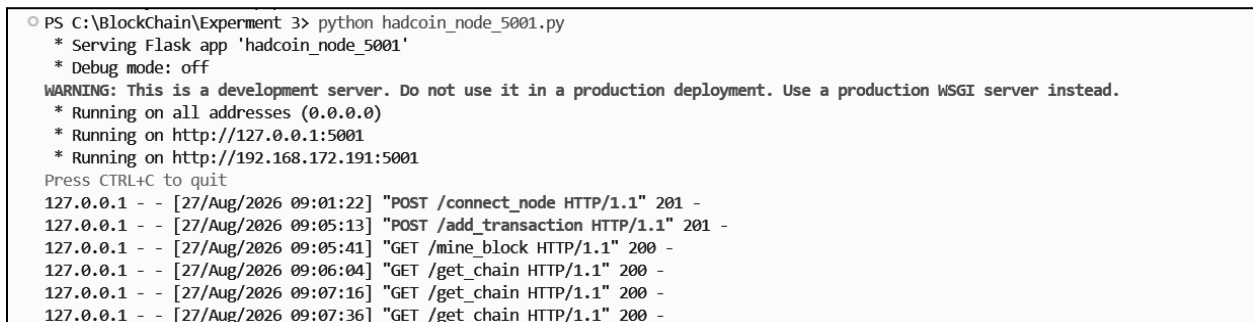
Step 1 :- downloading Lab 3 folder and setting it up on vscode



Step 2:- Installing necessary libraries



Step 3:- Running hadcoin_node_5001, hadcoin_node_5002, hadcoin_node_5003 on separate terminals



```
> Terminal
PS C:\BlockChain\Experiment 3> python hadcoin_node_5002.py
* Serving Flask app 'hadcoin_node_5002'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5002
* Running on http://192.168.172.191:5002
Press CTRL+C to quit
127.0.0.1 - - [27/Aug/2026 09:02:32] "POST /connect_node HTTP/1.1" 201 -
127.0.0.1 - - [27/Aug/2026 09:06:21] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [27/Aug/2026 09:07:16] "GET /replace_chain HTTP/1.1" 200 -
127.0.0.1 - - [27/Aug/2026 09:07:36] "GET /get_chain HTTP/1.1" 200 -
█
```

```
PS C:\BlockChain\Experiment 3> python hadcoin_node_5003.py
* Serving Flask app 'hadcoin_node_5003'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5003
* Running on http://192.168.172.191:5003
Press CTRL+C to quit
127.0.0.1 - - [27/Aug/2026 09:03:03] "POST /connect_node HTTP/1.1" 201 -
127.0.0.1 - - [27/Aug/2026 09:06:35] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [27/Aug/2026 09:07:16] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [27/Aug/2026 09:07:36] "GET /replace_chain HTTP/1.1" 200 -
█
```

Step 4:- Connect nodes using Postman

The screenshot shows the Postman interface with a new collection named '5001'. A POST request is configured to 'http://127.0.0.1:5001/connect_node'. The body is set to 'raw' (JSON) with the following content:

```
{
  "nodes": ["http://127.0.0.1:5002", "http://127.0.0.1:5003"]
}
```

The response is displayed in the 'Body' tab, showing a 201 CREATED status with a response time of 27 ms and a body size of 325 B. The JSON response is:

```
{
  "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
  "total_nodes": [
    "127.0.0.1:5002",
    "127.0.0.1:5003"
  ]
}
```

New Collection / 5002

POST http://127.0.0.1:5002/connect_node

Send

Docs Params Authorization Headers (10) Body Scripts Settings Cookies

Key	Value	Description
☒ Content-Type	application/json	

Body Cookies Headers (5) Test Results

201 CREATED • 8 ms • 325 B

Save Response

JSON

```
1 {
2   "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
3   "total_nodes": [
4     "127.0.0.1:5001",
5     "127.0.0.1:5003"
6   ]
7 }
```

New Collection / 5003

POST http://127.0.0.1:5003/connect_node

Send

Docs Params Authorization Headers (10) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON

Schema Beautify

```
1 {
2   "nodes": ["http://127.0.0.1:5001", "http://127.0.0.1:5002"]
3 }
```

Body Cookies Headers (5) Test Results

201 CREATED • 14 ms • 325 B

Save Response

JSON

```
1 {
2   "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
3   "total_nodes": [
4     "127.0.0.1:5001",
5     "127.0.0.1:5002"
6   ]
7 }
```

Step 5:- Add Transaction to Node 5001

New Collection / Transactions

POST http://127.0.0.1:5001/add_transaction

Send

Docs Params Authorization Headers (10) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON

Schema Beautify

```
1 {
2   "sender": "a",
3   "receiver": "b",
4   "amount": 51
5 }
6
```

Body Cookies Headers (5) Test Results

201 CREATED • 7 ms • 226 B

Save Response

JSON

```
1 {
2   "message": "This transaction will be added to Block 2"
3 }
```

Step 6:- Mining Transactions on Node 5001



HTTP New Collection / Mining a Block

GET http://127.0.0.1:5001/mine_block Send

Docs Params Authorization Headers (7) Body Scripts Settings Cookies

Query Params

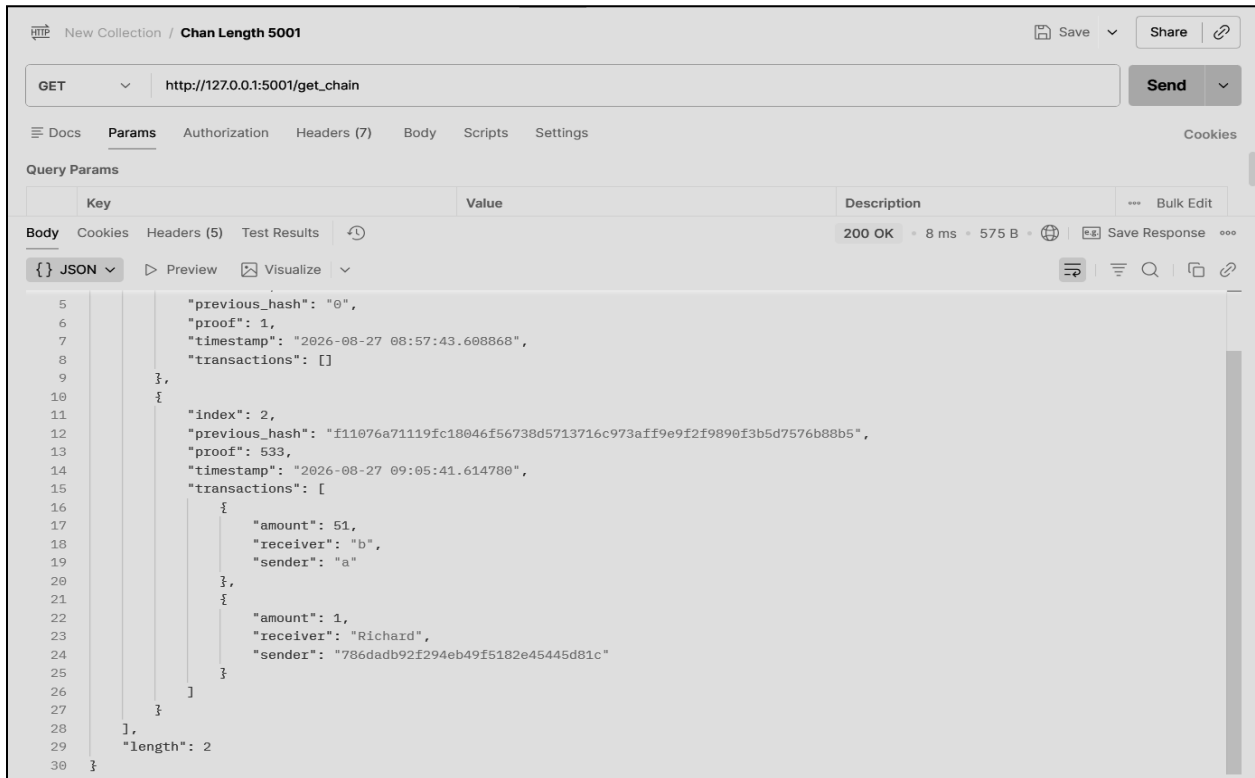
Key	Value	Description
-----	-------	-------------

Body Cookies Headers (5) Test Results 200 OK • 12 ms • 504 B Save Response

{ } JSON Preview Visualize

```
1 {
2   "index": 2,
3   "message": "Congratulations, you just mined a block!",
4   "previous_hash": "f11076a71119fc18046f56738d5713716c973aff9e9f2f9890f3b5d7576b88b5",
5   "proof": 533,
6   "timestamp": "2026-08-27 09:05:41.614780",
7   "transactions": [
8     {
9       "amount": 51,
10      "receiver": "b",
11      "sender": "a"
12    },
13    {
14      "amount": 1,
15      "receiver": "Richard",
16      "sender": "786dad92f294eb49f5182e45445d81c"
17    }
18  ]
19 }
```

Step 7:- Verifying length of each node



HTTP New Collection / Chan Length 5001

GET http://127.0.0.1:5001/get_chain Send

Docs Params Authorization Headers (7) Body Scripts Settings Cookies

Query Params

Key	Value	Description
-----	-------	-------------

Body Cookies Headers (5) Test Results 200 OK • 8 ms • 575 B Save Response

{ } JSON Preview Visualize

```
5   "previous_hash": "0",
6   "proof": 1,
7   "timestamp": "2026-08-27 08:57:43.608868",
8   "transactions": []
9 },
10 {
11   "index": 2,
12   "previous_hash": "f11076a71119fc18046f56738d5713716c973aff9e9f2f9890f3b5d7576b88b5",
13   "proof": 533,
14   "timestamp": "2026-08-27 09:05:41.614780",
15   "transactions": [
16     {
17       "amount": 51,
18       "receiver": "b",
19       "sender": "a"
20     },
21     {
22       "amount": 1,
23       "receiver": "Richard",
24       "sender": "786dad92f294eb49f5182e45445d81c"
25     }
26   ]
27 },
28 ],
29 "length": 2
30 }
```

Note :- you can observe the node on which performed the transaction, its length has increased by 1.

HTTP New Collection / Chain Length 5002

GET http://127.0.0.1:5002/get_chain

Send

Docs Params Authorization Headers (7) Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
-----	-------	-------------	-----------

Body Cookies Headers (5) Test Results

200 OK 8 ms 290 B Save Response

{ } JSON Preview Visualize

```
1 {
2   "chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-08-27 08:58:13.234187",
8       "transactions": []
9     }
10  ],
11  "length": 1
12 }
```

HTTP New Collection / Chain Length 5003

GET http://127.0.0.1:5003/get_chain

Send

Docs Params Authorization Headers (7) Body Scripts Settings Cookies

Body Cookies Headers (5) Test Results

200 OK 9 ms 290 B Save Response

{ } JSON Preview Visualize

```
1 {
2   "chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-08-27 08:58:26.195176",
8       "transactions": []
9     }
10  ],
11  "length": 1
12 }
```

Step 8:- Synchronizing the whole chain

REST client interface showing a GET request to `http://127.0.0.1:5002/replace_chain`. The response is a JSON object with status 200 OK, 18 ms, and 623 B. The JSON body is as follows:

```
1 {
2   "actual_chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-08-27 08:57:43.608868",
8       "transactions": []
9     },
10    {
11      "index": 2,
12      "previous_hash": "f11076a71119fc18046f56738d5713716c973aff9e9f2f9890f3b5d7576b88b5",
13      "proof": 533,
14      "timestamp": "2026-08-27 09:05:41.614780",
15      "transactions": [
16        {
17          "amount": 51,
18          "receiver": "b",
19          "sender": "a"
20        },
21        {
22          "amount": 1,
23          "receiver": "Richard",
24          "sender": "786dadb92f294eb49f5182e45445d81c"
25        }
26      ]
27    }
28  ],
29   "message": "All good. The chain is the largest one."
30 }
```

REST client interface showing a GET request to `http://127.0.0.1:5003/replace_chain`. The response is a JSON object with status 200 OK, 25 ms, and 657 B. The JSON body is as follows:

```
1 {
2   "message": "The nodes had different chains so the chain was replaced by the longest one.",
3   "new_chain": [
4     {
5       "index": 1,
6       "previous_hash": "0",
7       "proof": 1,
8       "timestamp": "2026-08-27 08:57:43.608868",
9       "transactions": []
10    },
11    {
12      "index": 2,
13      "previous_hash": "f11076a71119fc18046f56738d5713716c973aff9e9f2f9890f3b5d7576b88b5",
14      "proof": 533,
15      "timestamp": "2026-08-27 09:05:41.614780",
16      "transactions": [
17        {
18          "amount": 51,
19          "receiver": "b",
20          "sender": "a"
21        },
22        {
23          "amount": 1,
24          "receiver": "Richard",
25          "sender": "786dadb92f294eb49f5182e45445d81c"
26        }
27      ]
28    }
29  ]
30 }
```

Step 9 :- Verifying length of Node 5002 & 5003

GET `http://127.0.0.1:5002/get_chain` **Send**

Docs Params Authorization Headers (7) Body Scripts Settings Cookies

Body Cookies Headers (5) Test Results **200 OK** • 5 ms • 575 B • Save Response

JSON Preview Visualize

```
1 {
2   "chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-08-27 08:57:43.608868",
8       "transactions": []
9     },
10    {
11      "index": 2,
12      "previous_hash": "f11076a71119fc18046f56738d5713716c973aff9e9f2f9890f3b5d7576b88b5",
13      "proof": 533,
14      "timestamp": "2026-08-27 09:05:41.614780",
15      "transactions": [
16        {
17          "amount": 51,
18          "receiver": "b",
19          "sender": "a"
20        },
21        {
22          "amount": 1,
23          "receiver": "Richard",
24          "sender": "786dadb92f294eb49f5182e45445d81c"
25        }
26      ]
27    }
28  ],
29  "length": 2
30 }
```

GET `http://127.0.0.1:5003/get_chain` **Send**

Docs Params Authorization Headers (7) Body Scripts Settings Cookies

Body Cookies Headers (5) Test Results **200 OK** • 5 ms • 575 B • Save Response

JSON Preview Visualize

```
1 {
2   "chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-08-27 08:57:43.608868",
8       "transactions": []
9     },
10    {
11      "index": 2,
12      "previous_hash": "f11076a71119fc18046f56738d5713716c973aff9e9f2f9890f3b5d7576b88b5",
13      "proof": 533,
14      "timestamp": "2026-08-27 09:05:41.614780",
15      "transactions": [
16        {
17          "amount": 51,
18          "receiver": "b",
19          "sender": "a"
20        },
21        {
22          "amount": 1,
23          "receiver": "Richard",
24          "sender": "786dadb92f294eb49f5182e45445d81c"
25        }
26      ]
27    }
28  ],
29  "length": 2
30 }
```

Conclusion:

We developed a cryptocurrency using Python and implemented mining in a simulated three-node blockchain network. Each node maintained its own ledger and synchronized with peers using the Longest Chain Rule to ensure consistency. Mining was performed through Proof-of-Work, securely adding transactions and rewarding miners. This demonstrated key blockchain concepts, including decentralized consensus, transaction validation, and network synchronization.